6、驱动开发准备

全志V3S嵌入式驱动开发（驱动开发准备）

原创

嵌入式-老费Open: 7f185b501ef38a059b10d1b79d7cfdb9_MD5.png

已于 2024-06-28 21:17:04 修改

Open: 8a923f94c34ac151897c6be747452d7c_MD5.png

阅读量1.6k Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏 3

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

摘要 本文介绍了全志V3S驱动开发的准备工作，包括理解外部驱动芯片手册、研究SOC芯片手册、学习OS驱动框架、熟悉内核API以及编写上层代码。通过这些步骤，帮助读者掌握驱动外部硬件的基础知识和技能。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing @163.com】

之前的文章都是教大家怎么搭建环境、看原理图、编译内核和根文件系统、做镜像，直到现在才进入驱动开发的主题。 **毕竟整个专栏的目的，还是希望大家能够学会驱动外部硬件。** 驱动好硬件，分成硬件和软件两个部分，硬件的部分比较好理解，就是看相关的信号有没有对接上即可，但是软件对接、甚至于怎么写软件，大家好像接触的比较少，所以这个时候，就可以简单讨论一下，怎么样才能写驱动代码。

1、了解外部驱动芯片手册

所谓驱动，其实就是用soc里面的io控制器来驱动外部电路，或者说是外部芯片。如果是简单的io和uart这些，当然不需要知道驱动芯片的内容。但是如果驱动的是外部芯片，那么这个时候我们就要把对应芯片的资料找过来，好好读一读。里面协议的内容，就是我们将来编程的依据，

Open: 9e0e5eb9816e8283abd63efa6d6fc337_MD5.png

DAVICOM

DM9000

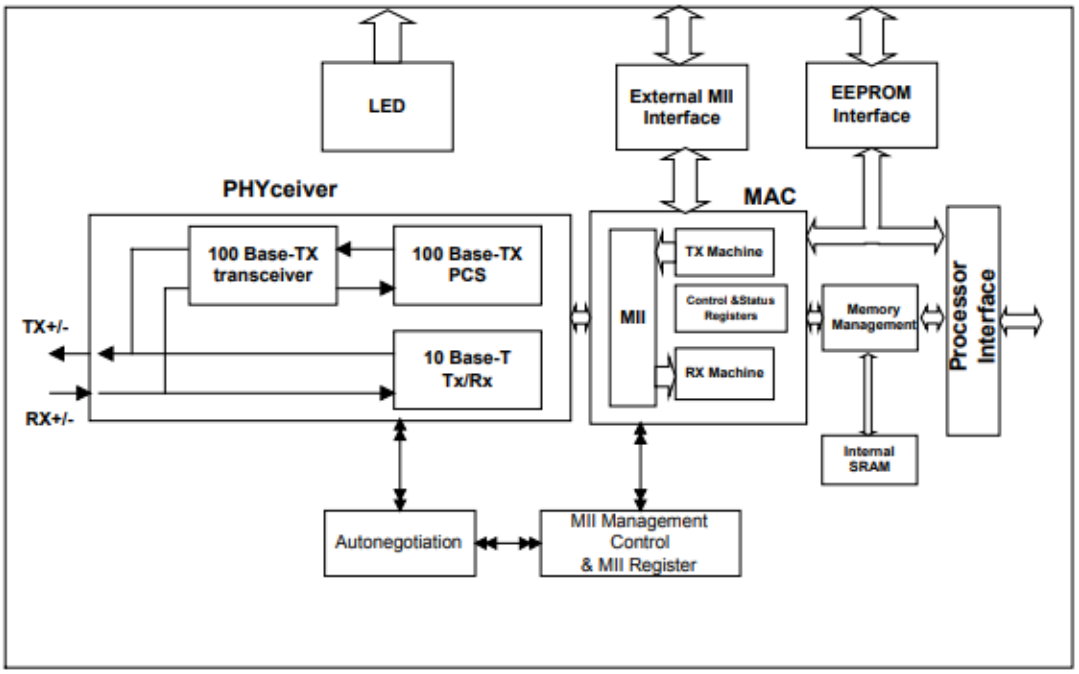
ISA to Ethernet MAC Controller with Integrated 10/100 PHY

1. General Description

The DM9000 is a fully integrated and cost-effective single chip Fast Ethernet MAC controller with a general processor interface, a 10/100M PHY and 4K Dword SRAM. It is designed with low power and high performance process that support 3.3V with 5V tolerance.

The DM9000 also provides a MII interface to connect HPNA device or other transceivers that support MII interface. The DM9000 supports 8-bit, 16-bit and 32-bit uP interfaces to internal memory accesses for different processors. The PHY of the DM9000 can interface to the UTP3, 4, 5 in 10Base-T and UTP5 in 100Base-TX. It is fully compliant with the IEEE 802.3u Spec. Its auto-negotiation function will automatically configure the DM9000 to take the maximum advantage of its abilities. The DM9000 also supports IEEE 802.3x full- duplex flow control. This programming of the DM9000 is very simple, so user can port the software drivers to any system easily.

2. Block Diagram



CSDN @嵌入式-老费

2、了解soc芯片手册

了解外部芯片主要是解决发送内容的问题，而soc芯片则会告诉我们怎么发送这些内容。比如，它是iic接口，还是spi接口，还是普通的uart接口等等。这些内容，都可以通过soc的芯片手册找到，比如下面s3c2440中nandflash controller的介绍，

Open: 297946019d3e2dec940beb4c7a353ce9_MD5.png

6 NAND FLASH CONTORLLER

OVERVIEW

In recent times, NOR flash memory gets high in price while an SDRAM and a NAND flash memory is comparatively economical , motivating some users to execute the boot code on a NAND flash and execute the main code on an SDRAM.

S3C2440A boot code can be executed on an external NAND flash memory. In order to support NAND flash boot loader, the S3C2440A is equipped with an internal SRAM buffer called 'Steppingstone'. When booting, the first 4 KBytes of the NAND flash memory will be loaded into Steppingstone and the boot code loaded into Steppingstone will be executed.

Generally, the boot code will copy NAND flash content to SDRAM. Using hardware ECC, the NAND flash data validity will be checked. Upon the completion of the copy, the main program will be executed on the SDRAM.

FEATURES

1. **Auto boot:** The boot code is transferred into 4-kbytes Steppingstone during reset. After the transfer, the boot code will be executed on the Steppingstone.
2. **NAND Flash memory I/F:** Support 256Words, 512Bytes, 1KWords and 2KBytes Page.
3. **Software mode:** User can directly access NAND flash memory, *for example this feature can be used in read/erase/program NAND flash memory*.
4. **Interface:** 8 / 16-bit NAND flash memory interface bus.
5. **Hardware ECC generation,** detection and indication (Software correction).
6. **SFR I/F:** Support Little Endian Mode, Byte/half word/word access to Data and ECC Data register, and Word access to other registers
7. **SteppingStone I/F:** Support Little/Big Endian, Byte/half word/word access.
8. **The Steppingstone** 4-KB internal SRAM buffer can be used for another purpose after NAND flash booting.

CSDN @嵌入式-老费

3、os驱动框架的学习

不同的os，它的驱动框架其实是不一样的。 **比如说，windows有windows的驱动框架，linux有linux的驱动框架，android系统甚至于把一部分驱动代码放到了应用层，这都是有可能的。** 所以，对于驱动工程师来说，如果需要编写驱动，那么要做的就是熟悉对应系统的驱动框架。一般来说，大部分驱动代码都是差不多的，没有必要从0开始写，可以找一个差不多的模板代码，根据芯片手册改一下内容，基本就可以用了。

当然，除了框架之外，还有一些特殊的配置文件需要注意，以linux驱动为例，除了驱动框架之外，还需要学习一下dts文件，也就是设备树，

```
https://github.com/Lichee-Pi/linux/tree/zero-4.10.y/arch/arm/boot/dts
```

4、熟悉os kernel API

一般来说，编写驱动的时候，我们也会用到内核的API。比如说，需要申请内存，需要申请中断，需要访问io地址等等，这些操作都需要小心、谨慎地去做。因为，如果是上层去做开发，最多也就是程序闪退，出不了大错，但是底层驱动如果写错，会直接重启硬件，或者程序死锁，这些都是很常见的现象。所以在真正使用API的时候，一定要理解了之后再使用，驱动的一些故障、特别是一些低概率的故障还是比较难调试的。

5、适当编写上层代码

驱动写好了，是需要搭配上层代码进行配套测试的。 如果公司不是很大，这部分也是需要驱动工程师自己去完成的。另外从提高工作效率的角度来说，自己会写上层应用，也可以更好地发现问题、解决问题，省的拖到最后解决，只会让自己搞得措手不及。

注：

上面说的都是软件的工作，有时候驱动工程师还需要自己量电压、测信号。 不需要很懂，但是可以解决最低级的故障。根据2/8原则来说，我们会的硬件知识也许不多，但是掌握了常见的套路之后，它已经足够可以解决80%的问题。这样遇到难题，也不会受制于人，效率上不打折扣。